

VITyarthi - BUILD YOUR OWN PROJECT

CampusFlow

DSA-Based Academic Task and Course Planner

Project Report

Submitted by

Krishnapal Rajput

Registration No.: 25MIM10084

Academic Year: 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

1. Introduction

CampusFlow is a command-line academic planning system created to demonstrate practical use of Data Structures and Algorithms. The application combines task prioritization, prerequisite planning, study-resource search, and simple analytics. Instead of using DSA concepts only in isolated textbook examples, CampusFlow applies them to common student workflows such as deciding which task should be done first, checking a valid learning sequence, and locating study material quickly.

The project is intentionally designed as a terminal application. This keeps execution independent of a graphical interface and makes the project easy to run in automated evaluation environments. All runtime features use the Python standard library and local JSON files, so no database server, API key, or internet connection is required.

2. Problem Statement

Students often track deadlines, course dependencies, and learning resources in separate places. A normal to-do list does not consider algorithmic priority, a plain course list does not represent prerequisite relationships, and a folder of notes does not provide structured search. The problem is to build one lightweight system that organizes these three activities while clearly demonstrating the correct use of core DSA concepts.

CampusFlow addresses this problem through three major modules: a heap-based task scheduler, a graph-based prerequisite planner, and a searchable resource catalogue supported by sorting, binary search, linear search, and hash indexing.

3. Objectives

- Apply heap, graph, queue, hashing, searching, sorting, and counting concepts in a real-world student-oriented problem.
- Provide a fully command-line executable application with simple, predictable input and output.
- Persist task and resource information locally using JSON storage.
- Validate inputs and handle invalid states such as graph cycles and malformed task details.
- Maintain modular code with independent components that can be tested separately.
- Provide automated tests and clear project documentation for reproducibility.

4. Functional Requirements

ID	Requirement	Description
FR-1	Task Management	Add, list, complete, and delete academic tasks.
FR-2	Priority Scheduling	Generate a recommended pending-task order using a heap.
FR-3	Course Graph	Represent prerequisite links using a directed adjacency-list graph.
FR-4	Course Ordering	Generate a valid course sequence using topological sorting.
FR-5	Cycle Detection	Reject prerequisite graphs that contain cycles.
FR-6	Prerequisite Path	Find a path between courses

		using BFS.
FR-7	Resource Management	Add, list, search, retrieve, and delete study resources.
FR-8	Resource Search	Support title-prefix search and topic-text search.
FR-9	Analytics	Show task-subject and resource-type distributions as ASCII charts.
FR-10	Persistence	Save tasks and resources in JSON files between program runs.

5. Non-Functional Requirements

Category	Requirement
Performance	Graph traversals should run in $O(V + E)$; direct ID lookup should be average $O(1)$; heap scheduling should use logarithmic heap operations.
Usability	Commands should have readable names, help text, and terminal-friendly output.
Reliability	Corrupt or missing JSON files should fall back safely rather than crash during startup.
Maintainability	Business logic, validation, storage, models, analytics, CLI, and tests should remain separated into modules.
Error Handling	Invalid dates, priorities, hours, empty values, missing IDs, self-prerequisites, and graph cycles should be handled explicitly.
Portability	The runtime should depend only on Python 3.10+ standard library features and work without a GUI.
Resource Efficiency	Data is stored in compact JSON files and loaded only into lightweight Python structures.
Testability	Core algorithmic classes should be callable directly without requiring terminal interaction.

6. System Architecture

CampusFlow uses a layered modular architecture. The CLI parses commands and delegates work to the appropriate module. Task and resource modules use a shared storage abstraction. The course graph remains in-memory because it represents a sample prerequisite structure. Analytics consumes task/resource objects and produces terminal summaries.

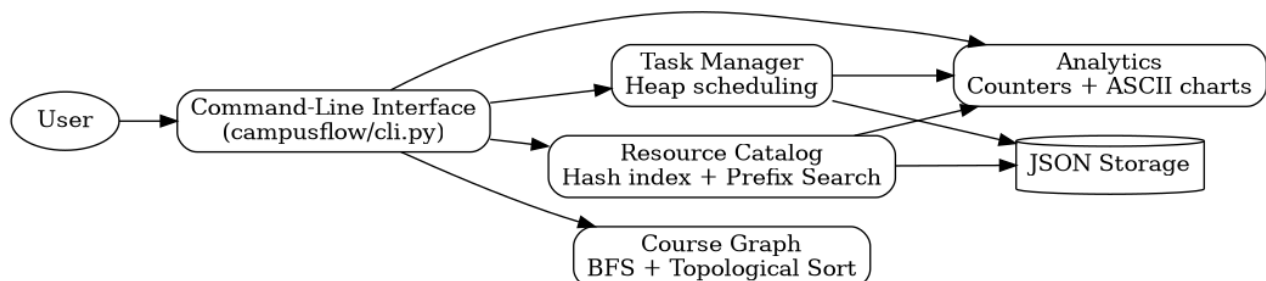


Figure 1: System Architecture

7. Design Diagrams

7.1 Use Case Diagram

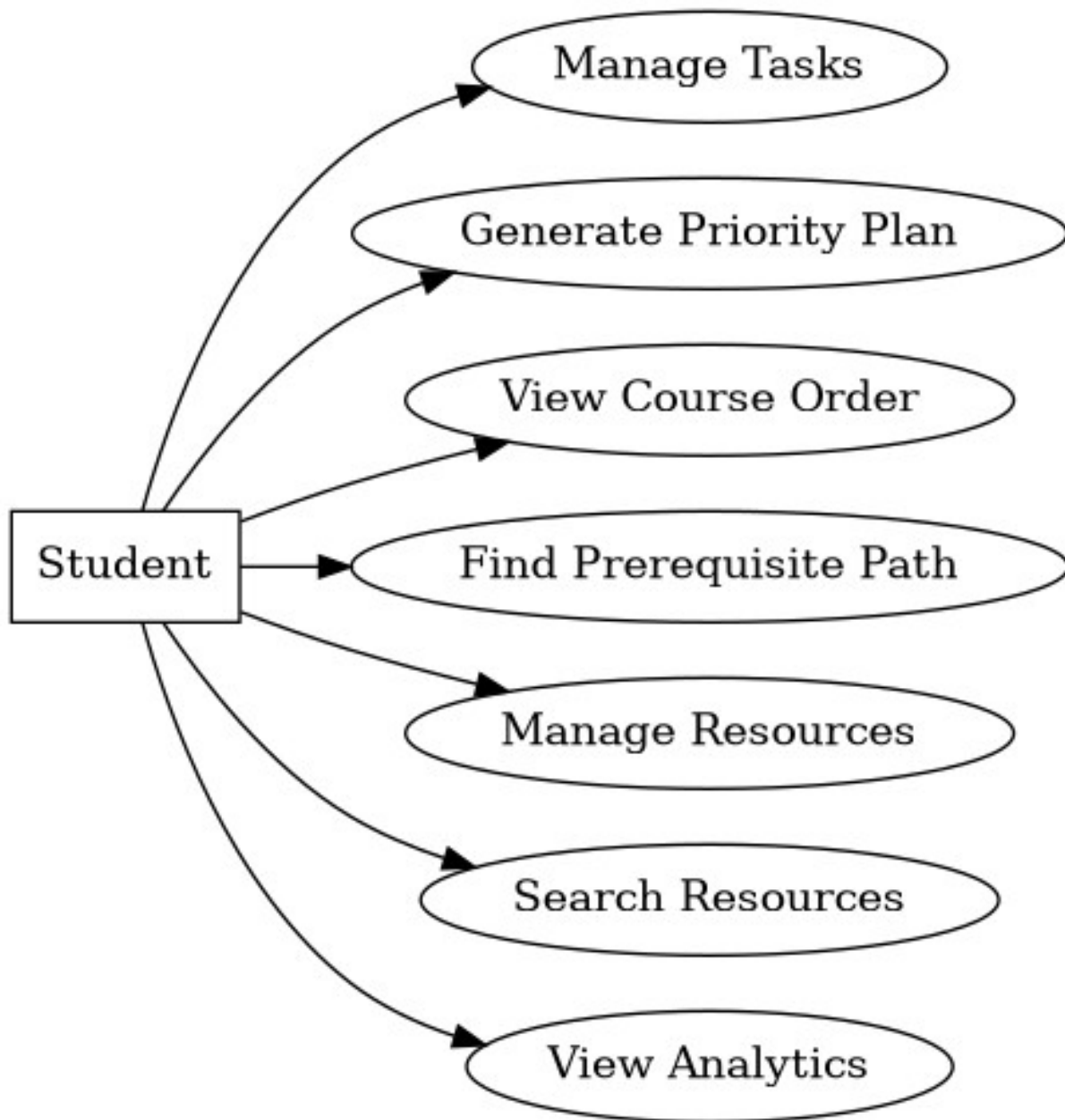


Figure 2: Use Case Diagram

7.2 Workflow Diagram

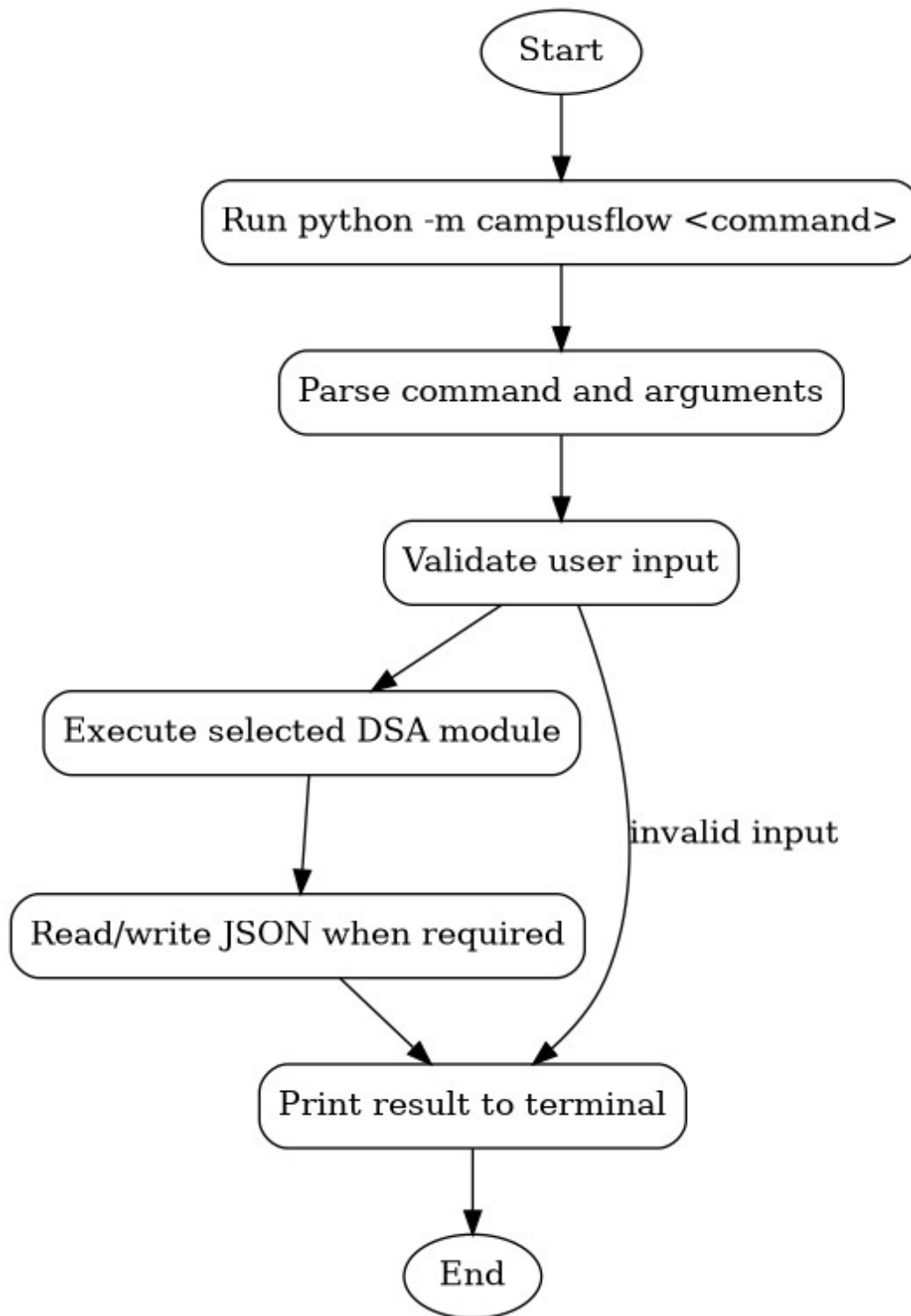


Figure 3: Overall Workflow

7.3 Class / Component Diagram

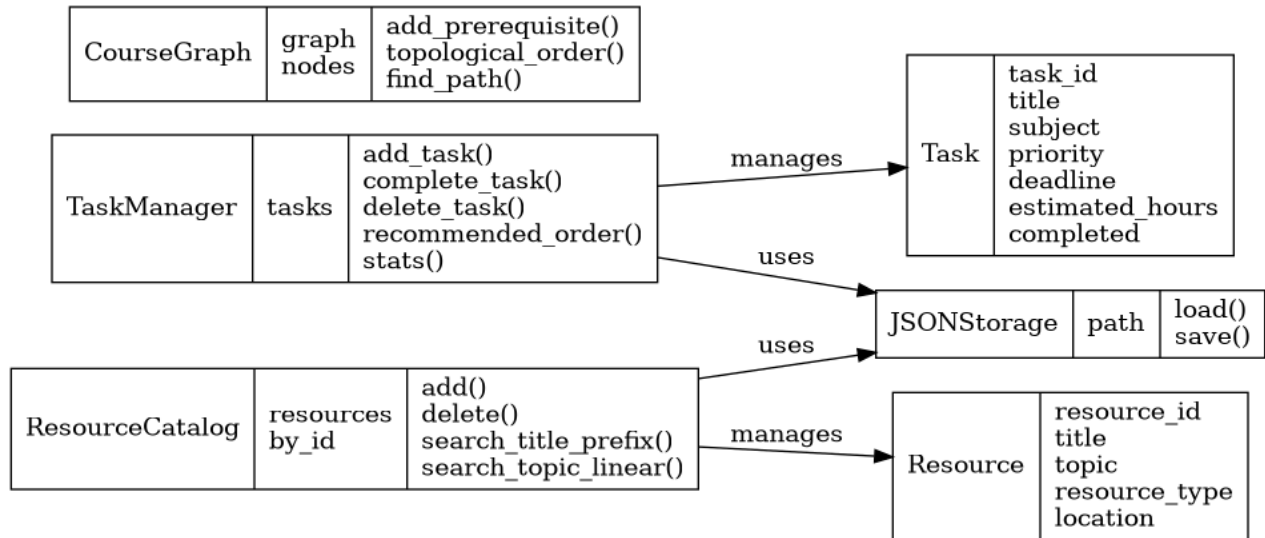


Figure 4: Class and Component Relationships

7.4 Sequence Diagram

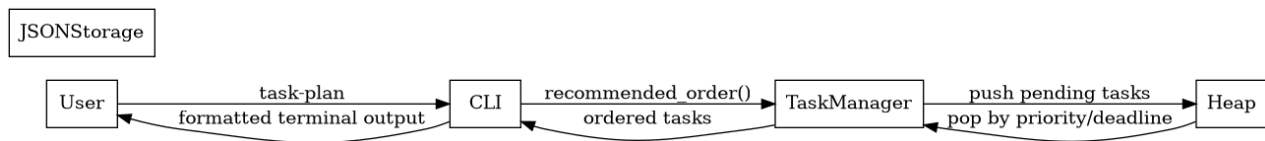


Figure 5: Sequence for Heap-Based Task Planning

7.5 Storage Schema

The project uses two small JSON documents. Their logical schemas are shown below.

File	Root Key	Important Fields
data/tasks.json	tasks	task_id, title, subject, priority, deadline, estimated_hours, completed
data/resources.json	resources	resource_id, title, topic, resource_type, location

8. Design Decisions & Rationale

Decision	Rationale
Binary heap for recommended task order	A heap is suited to repeatedly retrieving the next highest-priority item without fully re-sorting after every extraction.
Directed adjacency-list graph	Prerequisites are directional and most course graphs are sparse, making adjacency lists natural and memory-efficient.
Kahn's topological sort	It directly produces a valid prerequisite order and detects cycles by comparing processed-node count with total nodes.
BFS for prerequisite paths	The graph is unweighted; BFS finds a path with the fewest edges and is easy to reason about.

Dictionary for resource ID lookup	Python dict provides average $O(1)$ lookup and clean mapping from generated IDs to objects.
Sorted list + bisect for title prefix	Keeping resources sorted makes output predictable and permits $O(\log n)$ location of the first possible prefix match.
JSON instead of a DBMS	The dataset is small, local, portable, and does not justify an external database dependency.
Standard-library-only runtime	Reduces setup complexity and improves executability in automated terminal environments.

9. Implementation Details

9.1 Project Modules

Module	Responsibility
models.py	Task and Resource data models.
validators.py	Reusable validation for text, dates, priorities, and positive numeric values.
storage.py	JSON loading and safe replacement writes.
task_manager.py	Task CRUD, statistics, and heap-based scheduling.
course_graph.py	Directed graph, topological sorting, cycle detection, and BFS.
resource_catalog.py	Resource CRUD, sorted catalogue, hash index, binary-search prefix search, and linear topic search.
analytics.py	Counter-based summaries and ASCII bar charts.
sample_data.py	Repeatable sample dataset and sample course graph.
cli.py	Argument parsing, command routing, and terminal output.
__main__.py	Allows execution using <code>`python -m campusflow`</code> .

9.2 Algorithmic Complexity

Operation	Technique	Complexity
Task heap insertion	heapq.heappush	$O(\log n)$ each
Task heap extraction	heapq.heappop	$O(\log n)$ each
Topological sort	Queue + indegree on graph	$O(V + E)$
BFS path search	Queue traversal	$O(V + E)$
Resource ID lookup	Dictionary	Average $O(1)$
Prefix start location	bisect_left	$O(\log n)$
Topic search	Linear scan	$O(n)$
Analytics counting	Counter/hash table	$O(n)$

9.3 Input Validation and Error Handling

- Priority must be an integer from 1 to 5.
- Deadline must follow YYYY-MM-DD.
- Estimated hours must be greater than zero.
- Required text values cannot be blank.
- A course cannot depend on itself.
- Topological sorting raises an explicit error if a prerequisite cycle exists.
- Missing task/resource IDs return a readable not-found message.

- Malformed or unavailable JSON data falls back to an empty default structure.

10. Screenshots / Results

The following images were generated from actual command-line execution of the submitted project.

```
CampusFlow Terminal

$ python -m campusflow task-list
98a43213 | PENDING | P5 | 2026-09-20 | DSA | Revise linked lists | 2.0h
4863e316 | PENDING | P4 | 2026-09-22 | DSA | Practice graph traversal | 1.5h
3eec8925 | PENDING | P3 | 2026-09-24 | Programming Lab | Write lab record | 1.0h
```

Figure 6: Task Listing

```
CampusFlow Terminal

$ python -m campusflow task-plan
98a43213 | PENDING | P5 | 2026-09-20 | DSA | Revise linked lists | 2.0h
4863e316 | PENDING | P4 | 2026-09-22 | DSA | Practice graph traversal | 1.5h
3eec8925 | PENDING | P3 | 2026-09-24 | Programming Lab | Write lab record | 1.0h
```

Figure 7: Heap-Based Recommended Task Order

```
CampusFlow Terminal

$ python -m campusflow course-plan
Discrete Mathematics -> Programming Fundamentals -> Data Structures -> Algorithms -> Advanced Algorithms
```

Figure 8: Topological Course Order

```
CampusFlow Terminal

$ python -m campusflow course-path "Programming Fundamentals" "Advanced Algorithms"
Programming Fundamentals -> Data Structures -> Algorithms -> Advanced Algorithms
```

Figure 9: BFS Prerequisite Path

```
CampusFlow Terminal

$ python -m campusflow resource-search "Gra"
89b9d08f | Graph Algorithms Notes | BFS DFS shortest path | Notes | data/notes/graphs
```

Figure 10: Resource Prefix Search

```
CampusFlow Terminal

$ python -m campusflow analytics
Task distribution by subject
DSA ##### 2
Programming Lab ##### 1

Resource distribution by type
Notes ##### 1
Reference ##### 1
Worksheet ##### 1
```

Figure 11: Terminal Analytics

11. Testing Approach

Testing combines automated unit tests with a command-line smoke test. Unit tests create isolated temporary JSON files so the repository data is not modified. This makes the tests repeatable and independent.

Test Area	Cases Covered
Task Manager	Add, complete, heap ordering, invalid priority rejection
Course Graph	Topological ordering, cycle detection, BFS path
Resource Catalog	Title prefix search, hash ID lookup
Analytics	Empty dataset and normal chart formatting

Test command:

```
python -m unittest discover -s tests -v
```

Verified result during packaging: 10 tests executed successfully with final status OK.

12. Challenges Faced

- Combining multiple DSA concepts into one coherent application instead of creating unrelated demonstrations.
- Defining a task ordering rule that considers priority first while still using deadline and estimated time as deterministic tie-breakers.
- Making prerequisite validation detect cycles rather than silently returning a partial course order.
- Providing efficient prefix search while keeping the implementation simple and understandable.
- Keeping the project executable without third-party packages, databases, GUI libraries, or external services.

13. Learnings & Key Takeaways

- Data structures should be chosen according to the operation that matters most, not only by familiarity.

- Heaps are useful when repeated best-item selection matters more than maintaining a globally sorted structure at all times.
- Graphs naturally model dependencies; topological sort converts those dependencies into a usable order.
- BFS is a practical way to explore unweighted relationships level by level.
- Hash tables and binary search solve different lookup problems and can be combined in the same module.
- Modular architecture and tests make algorithmic code easier to verify and maintain.

14. Future Enhancements

- Allow users to persist and edit custom course graphs rather than using a sample graph.
- Add weighted scheduling that combines urgency, effort, and remaining days into a configurable score.
- Add Dijkstra/A* for weighted campus navigation as an optional fourth DSA module.
- Add CSV import/export for tasks and resources.
- Add benchmark commands to compare linear search, binary search, and hashing on larger generated datasets.
- Add optional SQLite storage while keeping JSON as a zero-configuration mode.
- Add richer terminal formatting while preserving non-GUI execution.

15. References

16. Python Software Foundation. Python 3 Standard Library documentation: heapq, collections, bisect, argparse, json, unittest.
17. Course concepts: heaps/priority queues, graph representations, breadth-first search, topological sorting, hashing, searching, sorting, and complexity analysis.
18. VITyarthi Build Your Own Project - General Project Instructions & Submission Guidelines (provided course document).